

Headless Mode

자동화와 CI/CD 통합

--no-interactive, KIRO_API_KEY, --trust-tools, --require-mcp-startup

GitHub Actions, 빌드 troubleshoot, 파이프라인 통합 패턴

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



Agenda

Chapter 16, Headless Mode

- 01 Headless Mode 개요
비대화형 실행, 활용 시나리오 4 가지
- 02 API Key 인증
KIRO_API_KEY 환경변수, 발급과 관리
- 03 4 가지 핵심 플래그
no-interactive, trust-all, trust, require-mcp
- 04 GitHub Actions 통합
PR 자동 리뷰 워크플로우 예시
- 05 기타 활용 패턴
테스트 생성, 빌드 troubleshoot, pipe 입력
- 06 보안 Best Practices
Secret 관리, 최소 권한, key rotation
- 07 제약사항과 Exit Code
TUI 비활성, slash 명령 제한

Headless Mode 개요

NON-INTERACTIVE EXECUTION

HEADLESS OVERVIEW

Headless mode 는 Kiro CLI 를 비대화형으로 실행하는 방식. API key 인증 후 prompt 를 인자로 전달하면 Kiro 가 end-to-end 로 실행. 브라우저 없는 환경, CI/CD 파이프라인, cron job, 컨테이너 빌드에 적합.

PR REVIEW

PR 자동 리뷰

GitHub Actions 통합.
매 PR 마다 보안, 품질, 테스트 커버리지 자동 분석.

TEST GENERATION

테스트 자동 생성

기능 코드 변경 시 단위 테스트 자동 작성과 실행.
커버리지 자동 유지.

TROUBLESHOOT

빌드 실패 분석

에러 로그 pipe 입력. 실패 원인 분석과 수정 제안.
CI 파이프라인 자가 진단.

AUTOMATION

반복 작업 자동화

Cron job 으로 정기 작업.
의존성 업데이트, 보안 audit, 문서 갱신 등.

Headless

비대화형 모드

--no-interactive

활성화 플래그

API key

인증 방식

CI/CD

주 활용 영역

API Key 인증

KIRO_API_KEY ENVIRONMENT VARIABLE

API KEY AUTHENTICATION

Headless mode 는 KIRO_API_KEY 환경변수로 인증. app.kiro.dev 의 API Keys 섹션에서 발급하며, 생성 시점에만 전체 키 값을 확인. Pro, Pro+, Power 구독자만 사용 가능.

GENERATION

Key 발급 절차

app.kiro.dev 로그인.
API Keys 섹션에서 새 키 생성.
명확한 이름 설정.
생성 시점에만 전체 키 확인 가능.

USAGE

환경변수 설정과 사용

```
export KIRO_API_KEY=key_xxx.
```

자동 인식되어 브라우저 인증 우회.
모든 기능 사용 가능.

GOVERNANCE

권한과 거버넌스

Pro/Pro+/Power 한정.
관리자가 API key 발급 활성화 필요.
MCP 제한, 모델 정책 동일 적용.

KIRO_API_KEY

환경변수 이름

Pro/Pro+/Power

구독 자격

Long-lived

Key 수명

app.kiro.dev

발급 사이트

4 가지 핵심 플래그

FLAGS THAT CONTROL HEADLESS MODE

HEADLESS FLAGS

Headless 모드의 동작은 4 가지 핵심 플래그로 제어.

--no-interactive 가 진입점, --trust-all-tools 와 --trust-tools 는 권한, --require-mcp-startup 는 MCP 의존성을 명시.

플래그	역할
--no-interactive	Headless 진입, prompt 인자 필요, 즉시 종료
--trust-all-tools	모든 도구 자동 승인, 광범위 권한
--trust-tools=cats	특정 도구 카테고리만 승인, 최소 권한 원칙
--require-mcp-startup	MCP 서버 실패 시 즉시 종료, fail-fast

4 플래그

Headless 핵심 옵션

Prompt arg

필수 입력 형태

Trust

권한 부여 방식

Fail fast

MCP 정책 옵션

GitHub Actions 통합

PR REVIEW WORKFLOW EXAMPLE

AUTOMATED PR REVIEW

GitHub Actions 와 Kiro CLI 의 통합은 가장 흔한 headless 활용 예시.

매 PR 마다 자동으로 코드 변경을 리뷰하고 결과를 의견으로 추가하는 워크플로우를 5 단계로 구성.

STEP 1

워크플로우 트리거

on: [pull_request].

PR 생성과 업데이트 마다 자동 실행.

main 브랜치 보호 권장.

STEP 2

Kiro CLI 설치

```
curl -fsSL  
https://cli.kiro.dev/install  
| bash.  
매 run 마다 fresh install.
```

STEP 3

Secret 주입

env: KIRO_API_KEY:
secrets.KIRO_API_KEY.
GitHub Secrets 에 미리
저장.

STEP 4

Kiro 실행

```
kiro-cli chat --no-  
interactive --trust-  
tools=read,grep with  
prompt.
```

GitHub Actions

주요 통합 대상

secrets.*

Secret 참조 형식

Per push

실행 주기

Fresh install

Runner 환경 정책

기타 활용 패턴

TEST GENERATION, TROUBLESHOOT, PIPE INPUT

ADDITIONAL HEADLESS PATTERNS

GitHub Actions PR 리뷰 외에도 headless mode 는 다양한 자동화에 활용.

테스트 자동 생성과 실행, 빌드 실패 분석, pipe 를 통한 컨텍스트 주입 3 가지 패턴이 가장 흔함.

TEST GEN

테스트 자동 생성과 실행

`kiro-cli chat --no-interactive --trust-all-tools` 와 함께 prompt 전달.
코드 분석, 테스트 작성, 실행, 결과 보고가 한 명령으로.

BUILD FIX

빌드 실패 분석

`cat build-error.log` 를 pipe 로 입력. Kiro 가 에러 분석 후 수정 제안.
컨테이너 빌드 자가 진단에 활용.

PIPE INPUT

Pipe 로 컨텍스트 주입

`git diff`, `log`, 데이터 파일 모두 pipe 가능.
풍부한 컨텍스트로 더 정확한 응답 생성.

Pipe

표준 입력 활용

stdout

응답 출력 채널

Cron

정기 실행 도구

End-to-end

한 명령 완료

보안 Best Practices

API KEY SAFETY AND LEAST PRIVILEGE

SECURITY PRINCIPLES

Headless mode 의 보안은 두 영역으로 나뉨.

API key 자체의 보안 관리, 그리고 실행 시 도구 권한의 최소화 원칙. 두 영역 모두 CI/CD 환경에서 필수.

SECRET MANAGEMENT

API Key 보안

1. CI/CD secret 으로만 저장, 절대 hardcode 금지
2. Source control 에 commit 금지
3. 환경변수 export 시 shell history 주의
4. 정기 rotation 과 사용 안 하면 즉시 revoke

LEAST PRIVILEGE

최소 권한 원칙

1. `--trust-tools` 로 카테고리 명시 권장
2. `--trust-all-tools` 는 꼭 필요할 때만
3. Readonly 작업은 `read`, `grep` 으로 충분
4. MCP 의존 시 `--require-mcp-startup` 추가

Secret

필수 저장 방식

Rotate

Key 정기 갱신

Least

권한 원칙

Exit code

실패 처리 체크

제약사항과 Exit Code

LIMITATIONS AND ERROR HANDLING

LIMITATIONS

Headless 제약사항 4 가지

1. Prompt 를 인자로 반드시 제공
2. Mid-session 사용자 입력 불가
3. Slash 명령 (/model, /agent picker) 불가
4. Terminal UI 기능 모두 비활성화

EXIT CODES

오류 처리 패턴

1. Exit code 로 파이프라인 실패 감지
2. Script 내 if 문으로 실패 처리 분기
3. MCP 실패는 `--require-mcp-startup` 와 결합
4. 공식 exit code 문서 참조 권장

Prompt 필수

기본 제약

No mid

중간 입력 불가

No TUI

UI 기능 비활성

Exit code

실패 감지 메커니즘

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

